

Annapurna Modern Data Stack - Question 1

Task-wise Outputs, Explanations, Reasons and Screenshots

This separate report documents Question 1, parts (a) to (f), using the available project outputs and screenshots.

Question 1(a): Project Setup and Data Processing

Task: Set up the project environment and process the source data using the pipeline.

Output:

Raw rows processed: 1,137,585
Duplicates removed: 16,661
Cleaned and transformed files generated.

Explanation: The pipeline read and processed the source data. Duplicate records were identified and removed before analytical processing.

Reason for the output: Duplicate removal prevents inflated counts and incorrect sales totals. Processing the data first ensures that later outputs use consistent records.

Question 1(b): PostgreSQL Database

Task: Load the operational and reference data into PostgreSQL and verify the tables.

Output:

Database: annapurna

stores: 12 rows
product_categories: 14 rows
products: 1,224 rows
price_revisions: 4,320 rows

Explanation: The PostgreSQL tables were created and populated with the required reference data.

Reason for the output: PostgreSQL is used for structured relational data. Checking row counts confirms that the loading process was successful.

Question 1(c): MinIO Object Storage

Task: Store raw, processed and generated files in the MinIO bucket.

Output:

Bucket: annapurna
Objects: 172
Approximate storage: 141.7 MiB

Explanation: The bucket contains raw files, processed datasets, reports and star-schema outputs.

Reason for the output: MinIO provides object storage for the data-lake layer and allows files to be stored separately from the relational database.

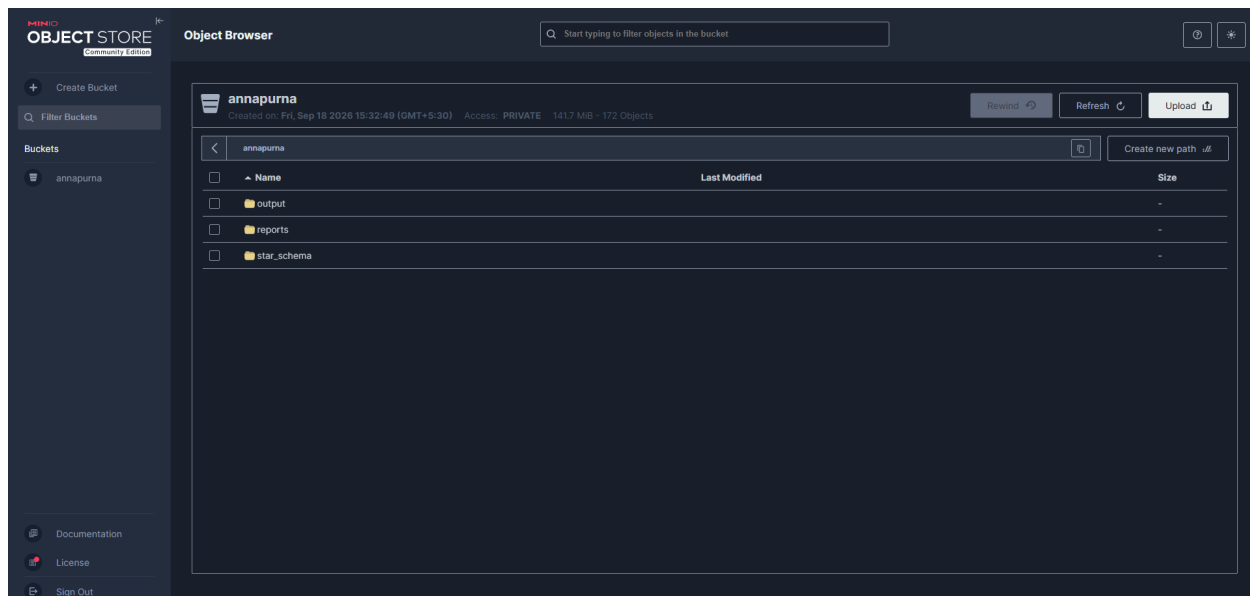


Figure: MinIO bucket and stored project objects

Question 1(d): DuckDB Analytical Processing

Task: Create the DuckDB database and generate analytical outputs.

Output:

DuckDB database: annapurna.duckdb

Generated files: daily_sales.csv, monthly_sales.csv, sales_cleaned.parquet and partitioned data.

Explanation: DuckDB processed the cleaned data and generated analytical summaries and a local analytical database.

Reason for the output: DuckDB is designed for analytical workloads and can query CSV and Parquet files efficiently.

Directory: C:\Users\ub02-glab-012\Downloads\Q1-Annapurna\output

Mode	LastWriteTime		Length	Name
----	-----	-----	-----	----
d-----	18-09-2026	15:27		partitioned
d-----	18-09-2026	15:27		star_schema
-a-----	18-09-2026	15:27	274432	annapurna.duckdb
-a-----	18-09-2026	15:32	11708	daily_sales.csv
-a-----	18-09-2026	15:32	430	monthly_sales.csv
-a-----	18-09-2026	15:27	209791	price_revisions.csv
-a-----	18-09-2026	15:27	102611	price_revisions.parquet
-a-----	18-09-2026	15:27	106629	products.csv
-a-----	18-09-2026	15:27	32616	products.parquet
-a-----	18-09-2026	15:27	472	product_categories.csv
-a-----	18-09-2026	15:27	3196	product_categories.parquet
-a-----	18-09-2026	15:32	21509063	sales_cleaned.parquet
-a-----	18-09-2026	15:27	1299	stores.csv
-a-----	18-09-2026	15:27	6044	stores.parquet

Figure: Generated project outputs

Question 1(e): ELT, Star Schema and Parquet

Task: Transform the data into curated analytical structures and Parquet files.

Output:

Generated outputs: stores.csv, products.csv, product_categories.csv, price_revisions.csv, sales_cleaned.parquet, partitioned/ and star_schema/.

Explanation: The outputs represent the curated data layer, including reference datasets, cleaned data, partitioned files and star-schema structures.

Reason for the output: A star schema separates facts from dimensions for analysis. Parquet provides columnar storage and is efficient for analytical queries.

```
Directory: C:\Users\ub02-glab-012\Downloads\Q1_Annapurna\final_submission
```

Mode	LastWriteTime		Length	Name
d-----	18-09-2026	15:43		partitioned
d-----	18-09-2026	15:43		star_schema
-a-----	18-09-2026	15:27	274432	annapurna.duckdb
-a-----	18-09-2026	15:32	11708	daily_sales.csv
-a-----	18-09-2026	15:32	278	finance_reconciliation.csv
-a-----	18-09-2026	15:32	430	monthly_sales.csv
-a-----	18-09-2026	15:27	209791	price_revisions.csv
-a-----	18-09-2026	15:27	106629	products.csv
-a-----	18-09-2026	15:27	472	product_categories.csv
-a-----	18-09-2026	15:32	21509063	sales_cleaned.parquet
-a-----	18-09-2026	15:27	1299	stores.csv

Figure: Final submission folder

Question 1(f): Finance Reconciliation and Final Submission

Task: Calculate monthly sales values, compare them with finance reference information and prepare the final submission.

Output:

Generated file: reports/finance_reconciliation.csv

Final submission includes the DuckDB database, CSV files, Parquet data, partitioned data and star_schema folder.

Explanation: The reconciliation file records calculated financial values. The final submission folder collects the main project deliverables.

Reason for the output: Reconciliation checks analytical calculations against available finance information. A single submission folder makes review and GitHub upload easier.

```
annapurna=# \q

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug annapurna\_postgre
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\ub02-glab-012\Downloads\Q1_Annapurna> Get-Content .\reports\finance_reconciliation.csv
month,calculated_revenue
2024-01,41629544.81
2024-02,37539932.03
2024-03,45168740.99
2024-04,40986594.69
2024-05,44865035.18
2024-06,42052907.32
2024-07,43292272.79
2024-08,48690619.05
2024-09,48161223.24
2024-10,60718437.18
2024-11,55888183.67
2024-12,54640017.72
PS C:\Users\ub02-glab-012\Downloads\Q1_Annapurna>
```

Figure: Finance reconciliation output

Conclusion

Question 1 covers data processing, PostgreSQL, MinIO, DuckDB, curated analytical outputs, star-schema and Parquet generation, and finance reconciliation. The screenshots provide evidence for the available project stages.